2_image_warping

2.1 IDW 算法

根据文档中的原理，在变换中我们需要计算的是矩阵 T，它们是线性方程组 $TA = B$ 的解。

而文档中也给出了 A, B 的计算方法，只需据此写出代码即可，同时由于存在一个求逆操作，因此我添加了一个正则项以避免求逆失败。

```
for (int i = 0; i < n; ++i)
{
    M2D A = M2D::Zero();
    M2D B = M2D::Zero();

    for (int j = 0; j < n; ++j)
    {
        if (i == j)
            continue;

        double sigma_ij = sigma(m_P[i], m_P[j]);

        V2D dp_ = m_P[j] - m_P[i];
        V2D dq_ = m_Q[j] - m_Q[i];

        // 跟据公式得出
        A += sigma_ij * dp_ * dp_.transpose();
        B += sigma_ij * dq_ * dp_.transpose();
    }

    // 添加正则项，避免求逆失败
    A += EPS * M2D::Identity();

    m_T[i] = B * A.inverse();
}
```

这里我自定义了几个常用类型名的简写，在 type_define.h

```cpp
#pragma once

#include <Eigen/Dense>

using namespace Eigen;

constexpr double INF = 1e12;
constexpr double EPS = 1e-12;

typedef Vector2d V2D;
typedef Matrix2d M2D;
```

在进行 warp 时，我们需要计算给定点的若干权重，并进行一个加权求和

```cpp
for (int i = 0; i < n_points_; ++i)
{
    sigma_array_[i] = sigma(p_, m_P[i]);
    sigma_sum_ += sigma_array_[i];
}

V2D q_ = V2D::Zero();
for (int i = 0; i < n_points_; ++i)
{
    double w_i = sigma_array_[i] / sigma_sum_;
    V2D f_i = m_Q[i] + m_T[i] * (p_ - m_P[i]);
    q_ += w_i * f_i;
}
```

2.2 RBF 算法

RBF 算法是一个线性变换 + 一个 R 函数的形式，我们在变换中需要计算的有线性变换的 A, b，以及 R 函数的参数 α。

这里我是用最小二乘方法实现的计算，即有

$$P * C = Q, C = (A, b)^T$$

使用 Eigen 自带的最小二乘方法 colPivHouseholderQr 即可求解

```
MatrixXd P_mat(n_points_, 3);
for (int i = 0; i < n_points_; ++i)
{
    P_mat(i, 0) = m_P[i].x();
    P_mat(i, 1) = m_P[i].y();
    P_mat(i, 2) = 1.0;
}

MatrixXd Q_mat(n_points_, 2);
for (int i = 0; i < n_points_; ++i)
{
    Q_mat(i, 0) = m_Q[i].x();
    Q_mat(i, 1) = m_Q[i].y();
}

// 最小二乘问题:  P_mat * C_mat = Q_mat
// C_mat = [A, b]^T
// => C_mat = (P^T P)^{-1} P^T Q
MatrixXd C_mat = P_mat.colPivHouseholderQr().solve(Q_mat);

A_(0, 0) = C_mat(0, 0);
A_(0, 1) = C_mat(1, 0);
A_(1, 0) = C_mat(0, 1);
A_(1, 1) = C_mat(1, 1);
b_(0) = C_mat(2, 0);
b_(1) = C_mat(2, 1);
```

得到 A, b 之后，求 α 就比较简单了，因为有等式

$$R_i = Q_i - (A * P_i + b) = \sum_{j=1}^{n} \alpha_j * g(||P_i - P_j||)\br => G * \alpha = R$$

其中 $G(i,j) = g(||P_i - P_j||)$，$R(i) = Q_i - (A * P_i + b)$，同样是一个线性方程组，同样使用 Eigen 自带的最小二乘方法 colPivHouseholderQr 求解

```
MatrixXd G(n_points_, n_points_);
MatrixXd R(n_points_, 2);

for (int i = 0; i < n_points_; ++i)
{
    // R_i = Q_i - (A*P_i + b)
    V2D R_i = m_Q[i] - (A_ * m_P[i] + b_);
    R(i, 0) = R_i.x();
    R(i, 1) = R_i.y();

    // 构造 G
    for (int j = 0; j < n_points_; ++j)        You, 14分钟
    {
        G(i, j) = g(m_P[i], m_P[j], r_[j]);
    }
}

// 求解 G * Alpha_mat = R
MatrixXd Alpha_mat = G.colPivHouseholderQr().solve(R);

for (int i = 0; i < n_points_; ++i)
{
    alpha_[i] = V2D(Alpha_mat(i, 0), Alpha_mat(i, 1));
}
```

2.3 特殊情况

在实现 RBF 的时候发现文档中考虑到用户有可能只给定 1 或 2 组控制点的情况，因此在 IDW 中补充了相应判断：

1. 1组控制点：显然是一个平移操作
2. 2组控制点：则是一个仿射变换，它包含旋转、缩放、平移三个操作，分别计算对应的参数即可

```
if (n_points_ == 2)
{
    V2D dp = m_P[0] - m_P[1];
    V2D dq = m_Q[0] - m_Q[1];
    double alpha = dq.norm() / (dp.norm() + EPS);
    double angle = std::atan2(dq.y(), dq.x()) - std::atan2(dp.y(), dp.x());
    M2D R;
    R << std::cos(angle), -std::sin(angle), std::sin(angle),
        std::cos(angle);
    m_T[0] = alpha * R;
    m_T[1] = alpha * R;
    return true;
}
```

2.4 补洞

出现空洞的原理就是由于变换后的点是连续的，而我们会将它们映射成离散点来设置像素颜色，因此这个映射并非一一映射，几乎必然存在空洞。

我一开始想的是确认一下变换后的边界与图像大小的交集作为补洞范围，然后查找没有被赋值的点，用插值来补一下。但是后来觉得这样比较难做，主要是因为这个边界很难计算。

后来想尝试逆变换，但是也比较难想。后来想到 Warper 求的是从源点到目标点的变换，而逆变换是从目标点到源点的变换。于是我直接交换了 Warper 的输入，然后遍历所有像素，找原图中的映射，如果映射存在，就直接填上，尽管可以用插值使得效果更好，但我觉得直接填上就差不多够用了。效果见 2.5 测试结果。

```
case kIDW:
{
    IDWWarper IDWWarper_inv(end_points_, start_points_);
    if (!IDWWarper_inv.get_transform())
    {
        std::cout << "IDW not implemented. Image is not warped."
                  << std::endl;
    }
    for (int y = 0; y < data_->height(); ++y)
    {
        for (int x = 0; x < data_->width(); ++x)
        {
            auto [src_x, src_y] = IDWWarper_inv.warp(x, y);
            if (src_x >= 0 && src_x < data_->width() && src_y >= 0 &&
                src_y < data_->height())
            {
                std::vector<unsigned char> pixel =
                    data_->get_pixel(src_x, src_y);
                warped_image.set_pixel(x, y, pixel);
            }
        }
    }
    break;
}
```

2.5 测试结果

2.3.1 IDW

$mu = 2$

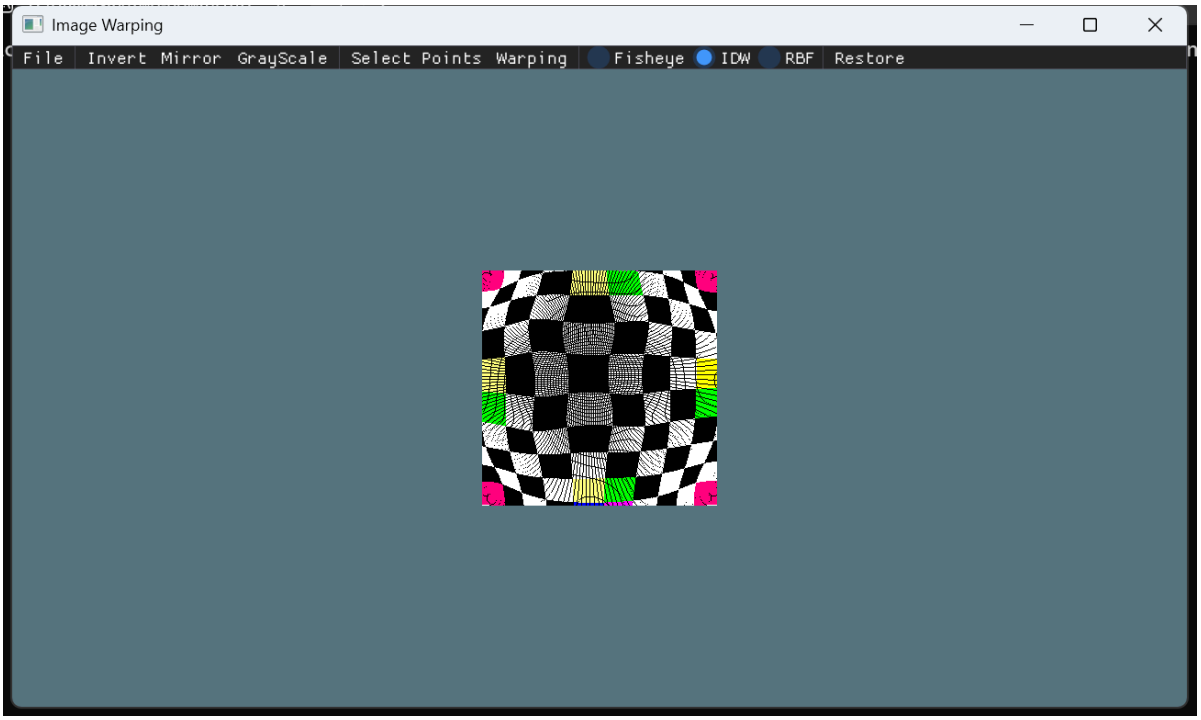


Image Warping
File
Invert
Mirror
GrayScale
Select Points
Warping
Fisheye
IDW
RBF
Restore

$mu = 3$

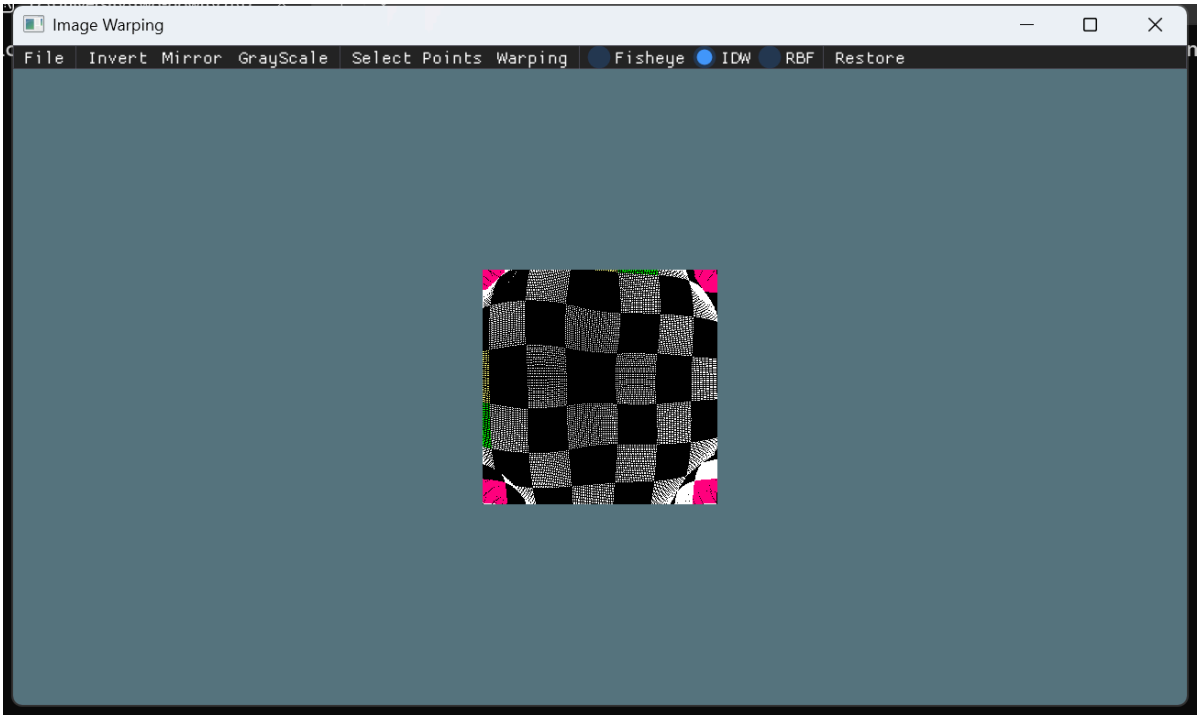
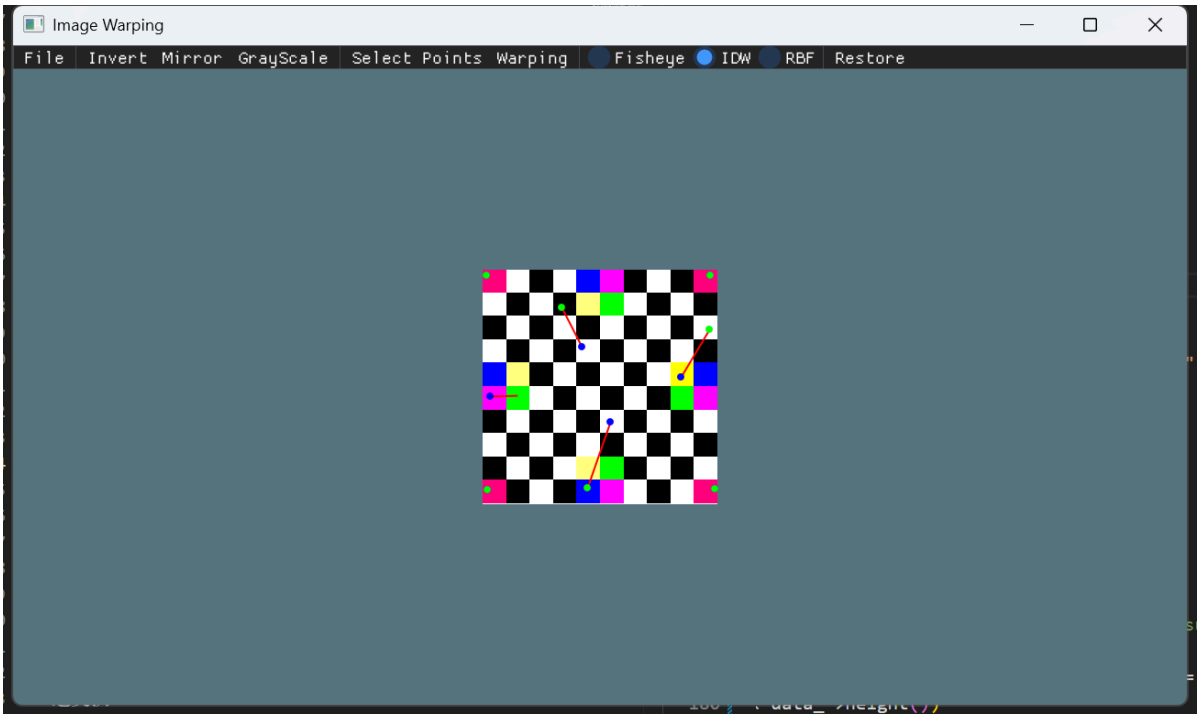
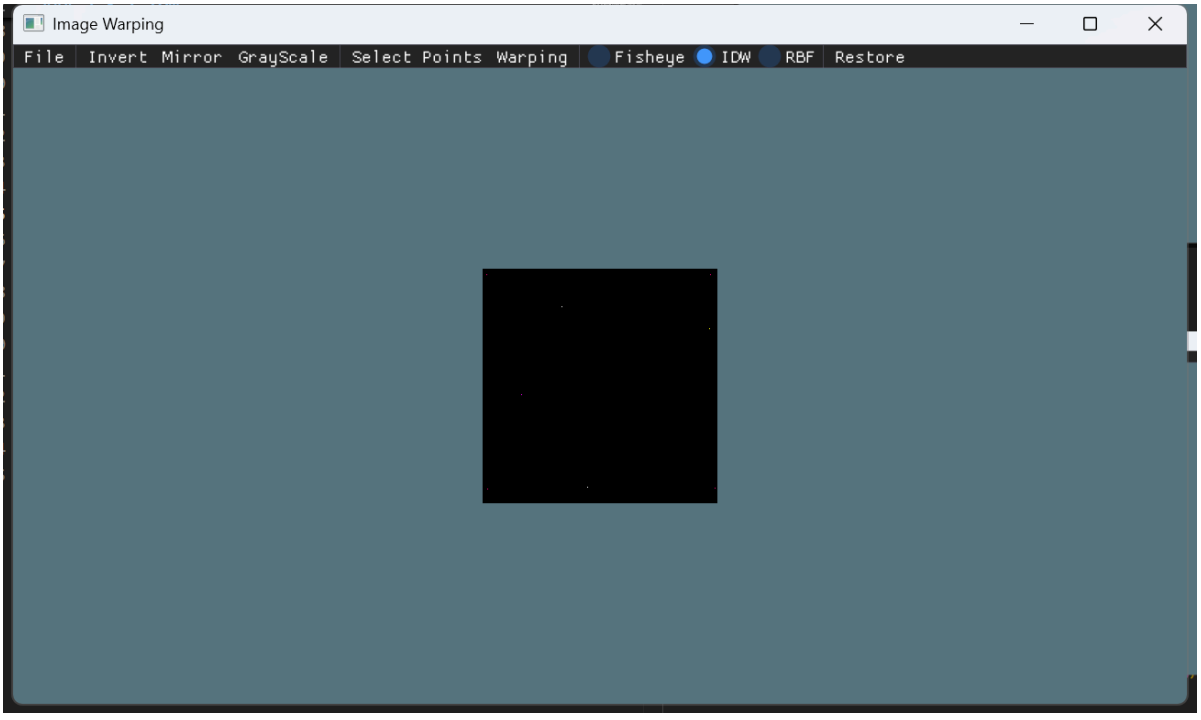


Image Warping
File
Invert
Mirror
GrayScale
Select Points
Warping
Fisheye
IDW
RBF
Restore

$mu = 1e12$





似乎 μ 越大，离控制点距离大小的影响越大，当 $\mu = 1e12$ 时，几乎可以看作是最近邻插值。

逆变换

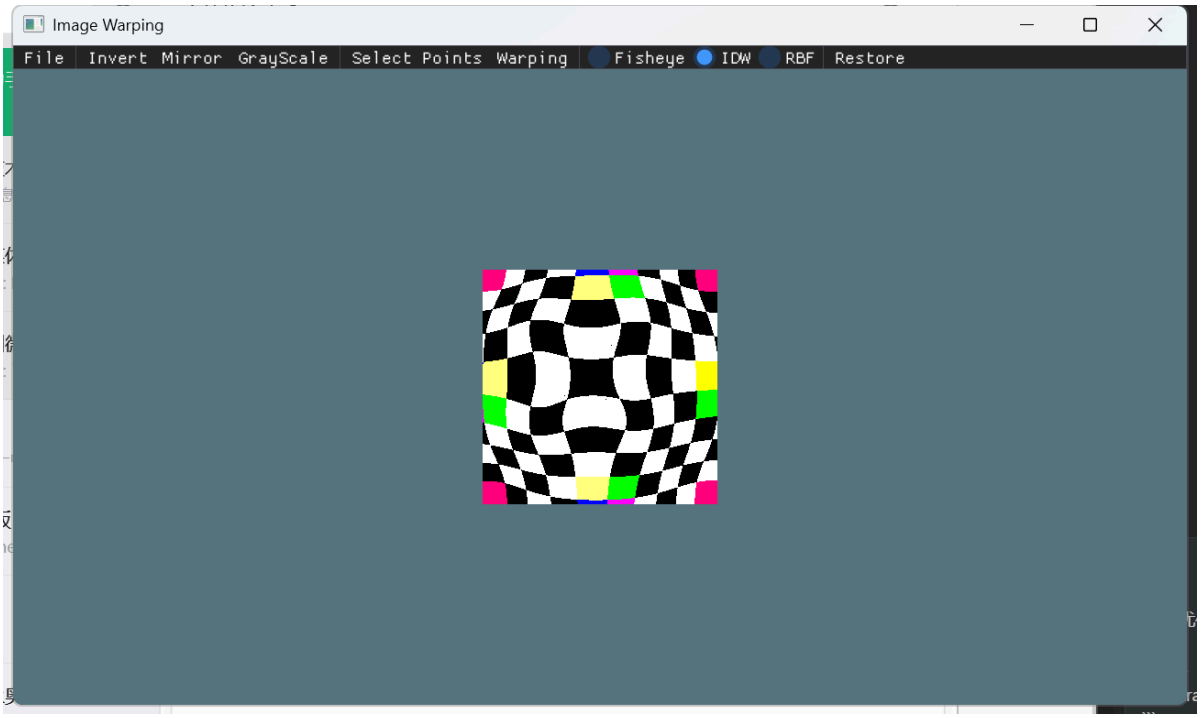


Image Warping
File
Invert
Mirror
GrayScale
Select Points
Warping
Fisheye
IDW
RBF
Restore

2.3.2 RBF

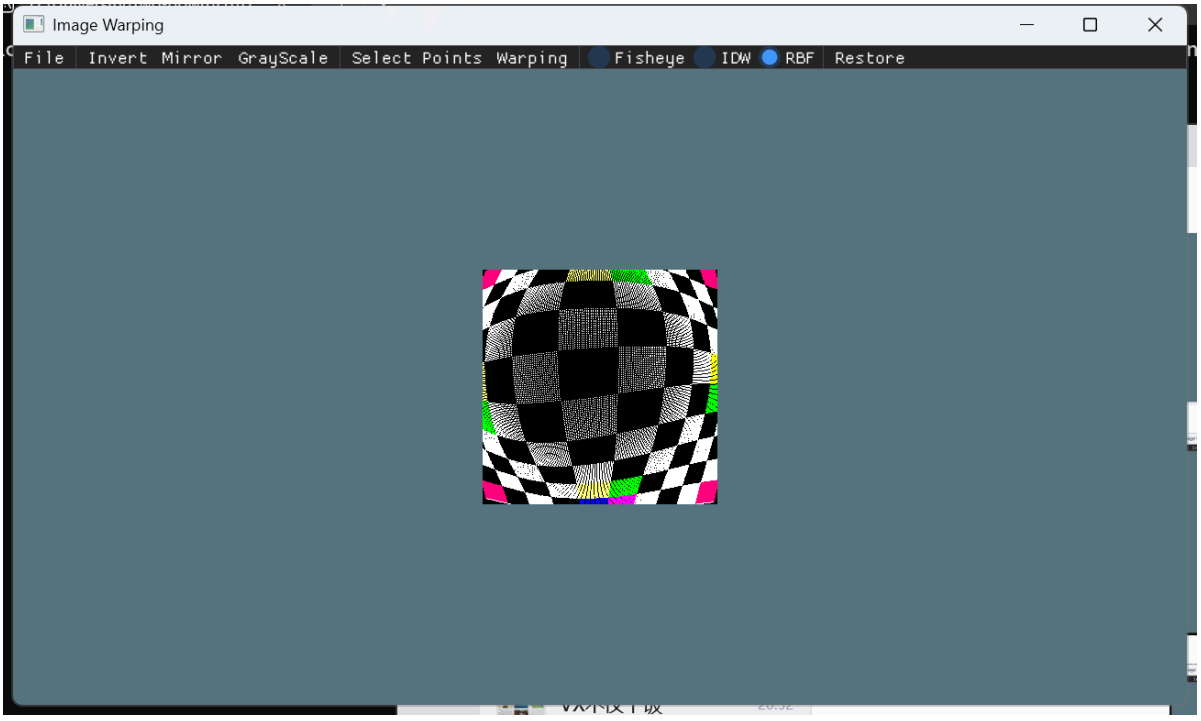


Image Warping
File
Invert
Mirror
GrayScale
Select Points
Warping
Fisheye
IDW
RBF
Restore

逆变换

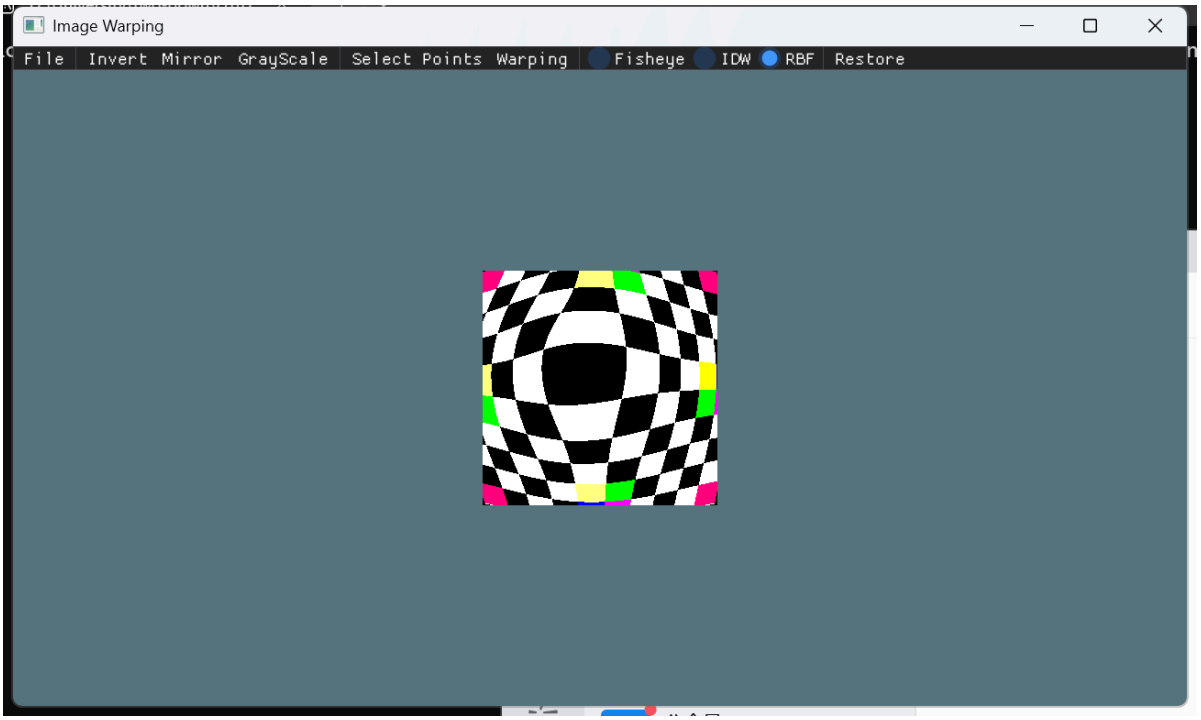


Image Warping
File
Invert
Mirror
GrayScale
Select Points
Warping
Fisheye
IDW
RBF
Restore

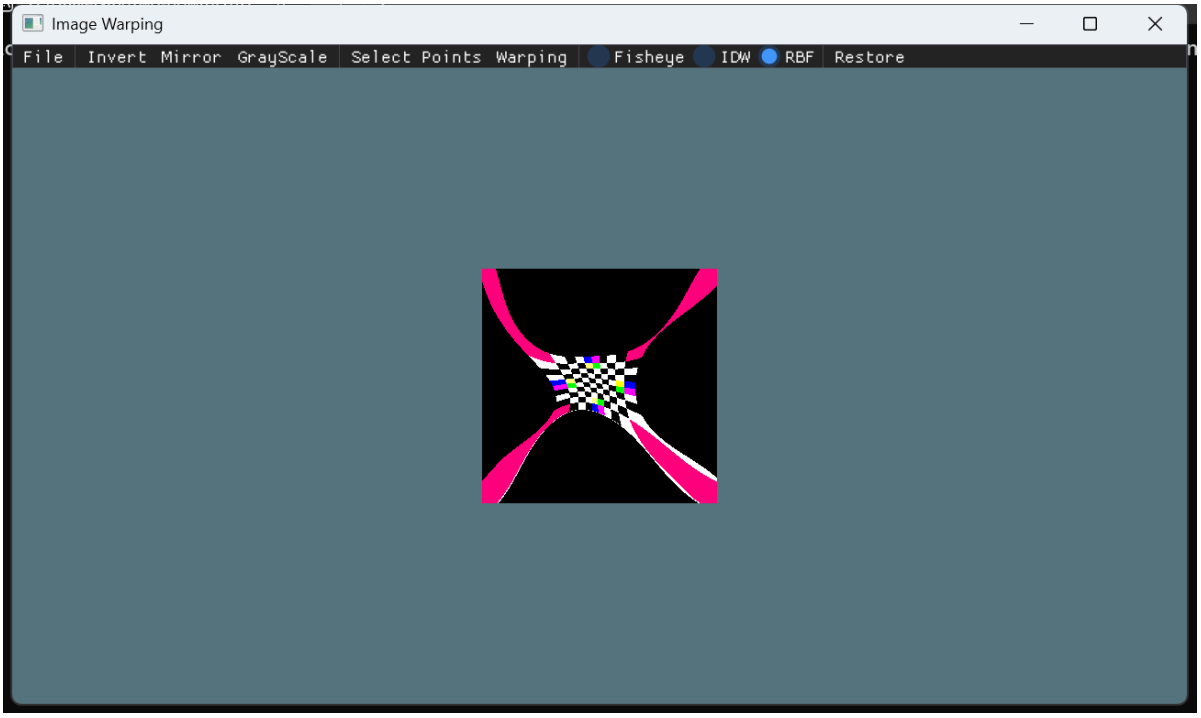


Image Warping
File
Invert
Mirror
GrayScale
Select Points
Warping
Fisheye
IDW
RBF
Restore

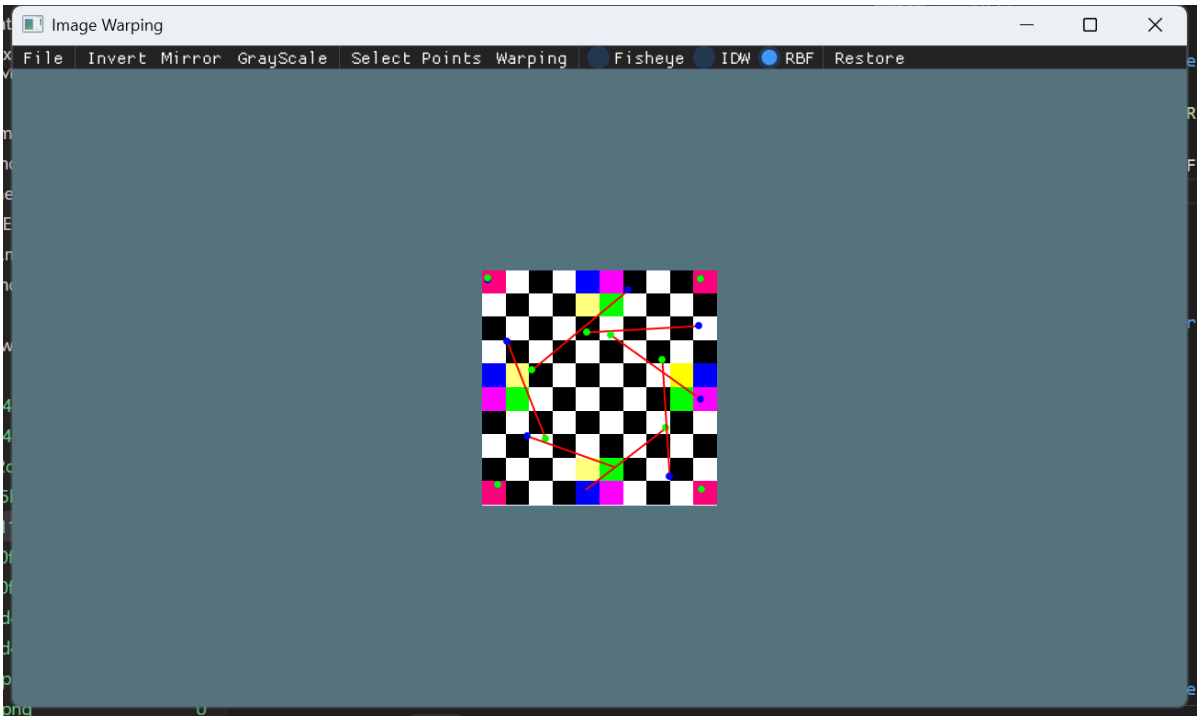


Image Warping
File
Invert
Mirror
GrayScale
Select Points
Warping
Fisheye
IDW
RBF
Restore

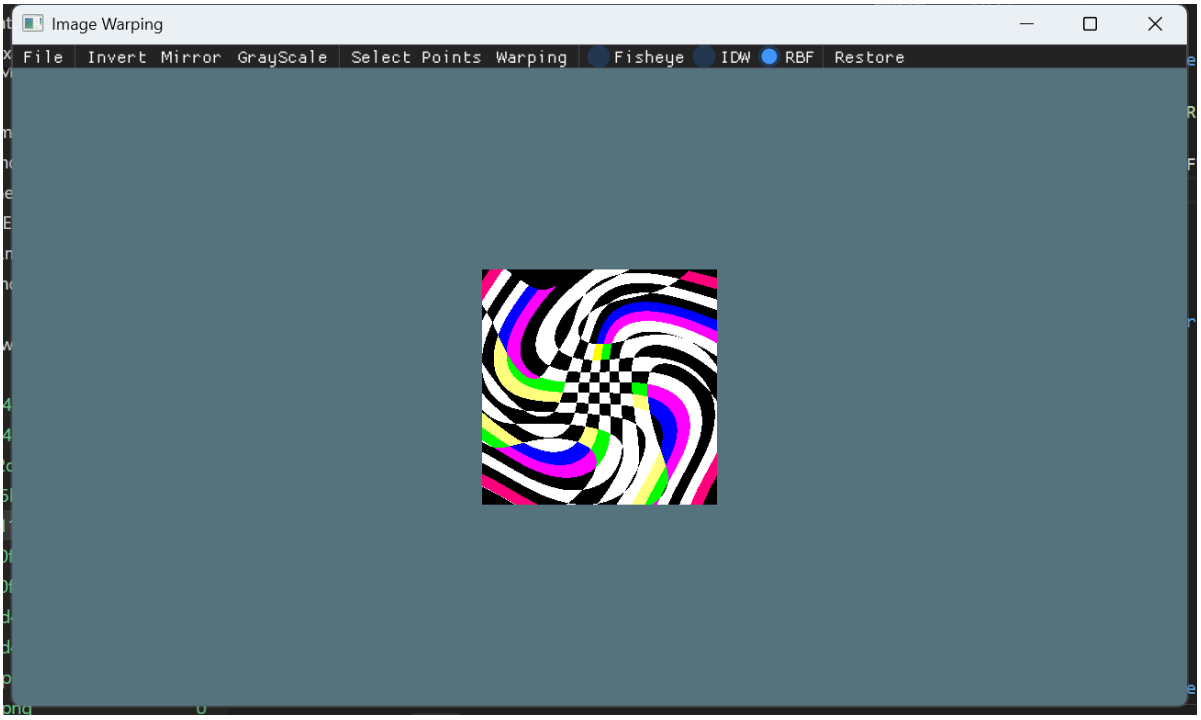


Image Warping
File
Invert
Mirror
GrayScale
Select Points
Warping
Fisheye
IDW
RBF
Restore